TypeScript

Union Types & the any Type

Definitions, Use Cases, Code Examples, Pitfalls, Best Practices and Interview Questions

Section	Topic
1	What are Union Types?
2	Why use Unions?
3	Narrowing Unions
4	Literal and Discriminated Unions
5	Unions with Arrays, Functions, Generics
6	Intersection vs Union
7	The any Type
8	any vs unknown vs never
9	Pitfalls and Best Practices
10	Interview Questions (30)
11	Quick Cheat Sheet

1. What is a Union Type?

A **union type** describes a value that can be **one of several types**. It is written with the pipe symbol **|** between the types. Read it as "this OR that".

```
let id: string | number;

id = "abc-123";   // OK
id = 42;          // OK
id = true;        // Error: Type 'boolean' is not assignable to type 'string | number'
```

A union only allows the members that are **common to every type** in the union, until you narrow it:

```
function show(value: string | number) {
  value.toString();     // OK  - both string and number have toString()
  value.toUpperCase();  // Error - number has no toUpperCase()
}
```

Where unions can be used

```
type ID = string | number;                 // type alias
interface User { id: ID; role: "admin" | "user"; }
function print(x: string | string[]) {}    // parameter
function parse(s: string): number | null { // return type
  const n = Number(s);
  return isNaN(n) ? null : n;
}
let maybe: string | undefined;             // optional-like values
```

2. Why are Unions Used?

- **Model real-world data** - an API may return a number or a string, a value or null.
- **Flexibility with safety** - more precise than any, since the compiler still checks every usage.
- **Restrict values** - literal unions like "GET" | "POST" replace magic strings and act like lightweight enums.
- **Represent states** - loading / success / error, with discriminated unions.
- **Better tooling** - autocomplete, exhaustive checks and refactoring support.
- **Handle nullability** - T | null | undefined makes missing values explicit (with strictNullChecks).

```
// Function accepting one or many items
function toArray(x: string | string[]): string[] {
  return Array.isArray(x) ? x : [x];
}

// Restricting allowed values
type Method = "GET" | "POST" | "PUT" | "DELETE";
function request(url: string, method: Method) {}
request("/users", "GET");     // OK
request("/users", "FETCH");   // Error
```

3. Narrowing Union Types

Narrowing means refining a union to a more specific type inside a code block. TypeScript's control-flow analysis does this automatically when you use one of the following techniques.

3.1 typeof guard

```
function format(v: string | number) {
```

```
  if (typeof v === "string") return v.toUpperCase(); // v: string
  return v.toFixed(2);                               // v: number
}
```

3.2 Truthiness and equality checks

```
function greet(name?: string | null) {
  if (!name) return "Hello, guest";   // handles undefined, null and ""
  return "Hello, " + name;            // name: string
}
function same(a: string | number, b: string | boolean) {
  if (a === b) { /* a and b are both string here */ }
}
```

3.3 instanceof guard

```
function log(e: Error | string) {
  if (e instanceof Error) console.log(e.message);
  else console.log(e);
}
```

3.4 in operator

```
type Bird = { fly(): void };
type Fish = { swim(): void };
function move(a: Bird | Fish) {
  if ("fly" in a) a.fly(); else a.swim();
}
```

3.5 Custom type guard (type predicate)

```
interface Cat { meow(): void }
interface Dog { bark(): void }

function isCat(p: Cat | Dog): p is Cat {
  return (p as Cat).meow !== undefined;
}
function talk(p: Cat | Dog) {
  if (isCat(p)) p.meow(); else p.bark();
}
```

3.6 Array.isArray and optional chaining

```
function len(x: string | string[] | undefined) {
  if (Array.isArray(x)) return x.length;  // string[]
  return x?.length ?? 0;                    // string | undefined
}
```

4. Literal and Discriminated Unions

4.1 Literal unions

```
type Direction = "up" | "down" | "left" | "right";
type DiceRoll = 1 | 2 | 3 | 4 | 5 | 6;
let d: Direction = "up";
```

4.2 Discriminated (tagged) unions

Every member has a common property (the **discriminant**) with a unique literal type. Checking it narrows the whole object. This is the most important real-world union pattern.

```
type Shape =
  | { kind: "circle"; radius: number }
  | { kind: "square"; side: number }
```

```
  | { kind: "rect"; w: number; h: number };

function area(s: Shape): number {
  switch (s.kind) {
    case "circle": return Math.PI * s.radius ** 2;
    case "square": return s.side ** 2;
    case "rect":   return s.w * s.h;
  }
}
```

4.3 Exhaustiveness checking with never

If you later add a new member to the union and forget to handle it, the compiler reports an error:

```
function assertNever(x: never): never {
  throw new Error("Unexpected value: " + JSON.stringify(x));
}
function area2(s: Shape): number {
  switch (s.kind) {
    case "circle": return Math.PI * s.radius ** 2;
    case "square": return s.side ** 2;
    case "rect":   return s.w * s.h;
    default:       return assertNever(s); // Error if a case is missing
  }
}
```

4.4 Real-world example: API state

```
type RequestState<T> =
  | { status: "loading" }
  | { status: "success"; data: T }
  | { status: "error"; error: string };

function render(state: RequestState<string[]>) {
  switch (state.status) {
    case "loading": return "Loading...";
    case "success": return state.data.join(", ");
    case "error":   return "Failed: " + state.error;
  }
}
```

5. Unions with Arrays, Functions and Generics

```
// Array of union  vs  union of arrays
let a: (string | number)[] = ["a", 1, "b", 2];  // each element may differ
let b: string[] | number[] = ["a", "b"];         // whole array is one type

// Union of function types
type Fn = ((x: string) => void) | ((x: number) => void);

// Generics with unions
function firstOr<T, D>(arr: T[], fallback: D): T | D {
  return arr.length ? arr[0] : fallback;
}
const r = firstOr([1, 2], "none");   // number | string

// Distributive conditional types over unions
type ToArray<T> = T extends any ? T[] : never;
type R = ToArray<string | number>;   // string[] | number[]

// Built-in utilities that work on unions
type Opt = string | null | undefined;
type Clean = NonNullable<Opt>;               // string
type OnlyStr = Extract<string | number, string>;  // string
type NoStr = Exclude<string | number, string>;    // number
```

Common union-related operations

Operation	Example	Result
Remove null/undefined	NonNullable<string | null>	string
Keep members	Extract<'a'|'b'|'c', 'a'|'b'>	'a' | 'b'
Remove members	Exclude<'a'|'b'|'c', 'a'>	'b' | 'c'
Keys as union	keyof {x:1; y:2}	'x' | 'y'
Values as union	(typeof obj)[keyof typeof obj]	union of values
From array	(typeof arr)[number]	union of element types (use as const)

```
const ROLES = ["admin", "editor", "viewer"] as const;
type Role = (typeof ROLES)[number];   // "admin" | "editor" | "viewer"
```

6. Union (|) vs Intersection (&)

Feature	Union A | B	Intersection A & B
Meaning	Value is A OR B	Value is A AND B
Accessible members	Only common members (before narrowing)	All members of both
Typical use	Alternatives, states	Combining/merging object types, mixins
Set theory	Wider set (grows)	Narrower set (shrinks)

```
type A = { name: string };
type B = { age: number };

type U = A | B;   // { name } OR { age }
type I = A & B;   // { name; age }

const i: I = { name: "Asha", age: 25 };  // needs both
```

7. The any Type

any is a special type that **turns off type checking** for a value. A variable of type any can hold anything, and you can call, index or assign it as if it were any other type, without compiler errors. It is TypeScript's "escape hatch" back to plain JavaScript behaviour.

```
let x: any = 10;
x = "hello";
x = { a: 1 };
x.foo.bar.baz();     // no compile error (crashes at runtime!)
x();                 // no error
const n: number = x; // no error, even if x is a string
```

Why is any used?

- **Migrating JavaScript to TypeScript** gradually, file by file.
- **Third-party libraries** with missing or poor type definitions.
- **Truly dynamic data** where the shape is unknown (e.g. JSON.parse result, though unknown is better).
- **Quick prototyping** or temporarily silencing a hard-to-type problem.

- **Implicit any** happens when a variable or parameter has no annotation and cannot be inferred (error under noImplicitAny).

Why is any dangerous?

- It **disables compile-time safety**: typos and wrong calls are not detected.
- It is **contagious**: anything derived from an any value is usually any too.
- It **kills autocomplete** and refactoring support.
- Runtime errors such as 'Cannot read properties of undefined' go unnoticed until production.

```
function double(v: any) { return v * 2; }
double("abc");   // NaN at runtime, no compile error

const data = JSON.parse('{"a":1}');   // data: any
data.b.c;                              // runtime TypeError, no warning
```

Where any appears implicitly

```
JSON.parse(text)            // returns any
response.json()             // Promise<any>
function f(x) {}            // x is implicit any (error with noImplicitAny)
catch (e) {}                // e was any (unknown with useUnknownInCatchVariables)
const arr = [];             // evolving any[] in some configs
```

Useful compiler options

Option	Effect
noImplicitAny	Errors when a type would silently become any (part of strict)
strict	Enables noImplicitAny, strictNullChecks and more
useUnknownInCatchVariables	catch (e) gives unknown instead of any
ESLint: no-explicit-any	Flags every written 'any'
ESLint: no-unsafe-*	Flags unsafe assignment/call/member access of any values

8. any vs unknown vs never vs union

Type	Assign anything in?	Use without checking?	Meaning / Use
any	Yes	Yes (unsafe)	Opt out of type checking
unknown	Yes	No - must narrow first	Safe 'top type' for untrusted data
never	No	n/a	Impossible values, exhaustive checks, functions that never return
string | number	Only those two	Only common members	Known set of alternatives

```
let a: any = "hi";
a.toUpperCase();          // allowed
a.nonExistent();          // allowed (runtime crash)

let u: unknown = "hi";
u.toUpperCase();          // Error: 'u' is of type 'unknown'
if (typeof u === "string") u.toUpperCase();  // OK after narrowing
```

```
function fail(msg: string): never { throw new Error(msg); }
```

Rule of thumb: prefer a **union** when you know the possible types, use **unknown** when you do not know the type, and use **any** only as a last resort.

Safely handling unknown data

```
function parse(json: string): unknown { return JSON.parse(json); }

const v = parse('{"name":"Ravi"}');
if (typeof v === "object" && v !== null && "name" in v) {
  console.log((v as { name: string }).name);
}

// Even better: validate with a schema library (e.g. Zod)
// const User = z.object({ name: z.string() });
// const user = User.parse(JSON.parse(text));
```

9. Pitfalls and Best Practices

Common union pitfalls

- **Accessing non-shared members** without narrowing causes errors.
- **Optional (?) vs | undefined**: an optional property can be omitted, while | undefined must still be provided (unless optional).
- **Truthiness bugs**: if (!x) treats 0, "" and false as missing. Use x == null or x === undefined for null-checks.
- **Array<A | B> vs A[] | B[]** behave differently.
- **Excess property checking** is looser with unions: an object literal may match a different member than intended.
- **Overlapping objects**: { a: 1; b: 2 } is assignable to { a: number } | { b: number } because unions of objects are inclusive.

```
type Props = { size?: number };       // may be omitted
type Props2 = { size: number | undefined }; // must be present

const p: Props = {};     // OK
const q: Props2 = {};    // Error: property 'size' is missing

function f(n: number | null) {
  if (!n) { /* also true when n is 0! */ }
  if (n === null) { /* correct null check */ }
}
```

Best practices

- Prefer **discriminated unions** over optional flags (isLoading?, error?) to prevent impossible states.
- Use **literal unions** or **as const** objects instead of enums for simple cases.
- Always add an **exhaustive never check** in switch statements over unions.
- Enable **strict** mode in tsconfig.json.
- Never write **any** in new code. Choose unknown, generics, or a specific union.
- Prefer **generics** when you want to preserve the relation between input and output types, instead of any.
- Keep unions **small and meaningful**; huge unions slow the compiler and confuse readers.
- Replace unavoidable any with a **narrow cast at the boundary** (e.g. validate API data once, then use real types).

Replacing any with better alternatives

```
// Bad
function identity(x: any): any { return x; }

// Good - generic keeps the type
function identity2<T>(x: T): T { return x; }

// Bad
function handle(input: any) { /* ... */ }

// Good - unknown forces checks
function handle2(input: unknown) {
  if (typeof input === "string") { /* ... */ }
}

// Bad
const cache: { [k: string]: any } = {};
// Good
const cache2: Record<string, unknown> = {};
```

10. Interview Questions and Answers

Basic Level

Q1. What is a union type in TypeScript?

A type formed from two or more types using |, meaning the value can be any one of them. Example: `string | number`.

Q2. What is the any type?

A type that disables type checking. Values of type any can be assigned anything and used in any way without compiler errors.

Q3. Why do we use union types?

To model values that may legitimately have multiple types while still getting compile-time safety, autocomplete and narrowing.

Q4. Why is any considered bad practice?

It removes type safety, spreads to derived values, hides bugs until runtime and degrades tooling. Prefer unknown, generics or unions.

Q5. Can you call string-only methods on string | number?

No. Only members common to both types are allowed until you narrow the type with typeof or another guard.

Q6. What is type narrowing?

Refining a broader type (like a union) to a more specific one within a code branch, using guards such as typeof, instanceof, in, equality and custom predicates.

Q7. How do you make a variable accept null in TypeScript?

Use a union: `let x: string | null`. With strictNullChecks, null and undefined are not assignable to other types otherwise.

Q8. What is a literal union type?

A union of specific literal values, such as `"GET" | "POST"`, limiting a variable to exactly those values.

Q9. What is implicit any?

When TypeScript cannot infer a type and none is given, it falls back to any. The noImplicitAny option turns this into an error.

Q10. Difference between string[] | number[] and (string | number)[]?

The first is either an all-string array or an all-number array. The second is a single array whose elements may each be string or number.

Intermediate Level

Q11. What is a discriminated union?

A union whose members share a literal-typed property (the discriminant, e.g. kind or type). Checking that property narrows the object to one member. Ideal for state machines and actions in reducers.

Q12. How do you ensure all union cases are handled?

Add a default branch that assigns the value to a never-typed variable or passes it to an assertNever function. A missing case produces a compile error.

Q13. any vs unknown?

Both accept every value. any allows unrestricted use, while unknown requires narrowing or assertion before use, so it is type-safe.

Q14. What is a type guard? Write one.

A function or expression that narrows types. Custom guards use a type predicate: `function isStr(x: unknown): x is string { return typeof x === "string"; }`

Q15. What does never mean and how does it relate to unions?

never is the empty type with no values. A union with never collapses: `string | never` is string. It is also the result when all union members are excluded or narrowed away.

Q16. What is the difference between union (|) and intersection (&)?

Union means one of the types; intersection means all types at once. Union widens the set of allowed values, intersection narrows it.

Q17. What does Exclude<T, U> do?

Removes from union T all members assignable to U. `Exclude<'a'|'b', 'a'>` gives 'b'.

Q18. How do you derive a union type from an array of values?

Use as const and indexed access: `const A = ['x','y'] as const; type T = (typeof A)[number];`

Q19. Why is the result of JSON.parse any, and how would you improve it?

JSON can contain anything, and the standard library types it as any. Improve by assigning to unknown and validating (manually or with Zod/io-ts) before use.

Q20. Is { a: 1; b: 2 } assignable to { a: number } | { b: number }?

Yes. Object union members are not exclusive; a value matching either (or both) is allowed. For exclusivity use a discriminant or mark the other property as never/undefined.

Advanced Level

Q21. What are distributive conditional types over unions?

When a conditional type is applied to a naked type parameter that is a union, it is evaluated for each member and the results are unioned. `T extends any ? T[] : never` with string | number gives string[] | number[]. Wrap in [T] to disable.

Q22. How does TypeScript treat union of function types when calling?

The parameters become an intersection. For `((x: string) => void) | ((x: number) => void)` you must pass something that is both string and number (effectively never).

Q23. What is the type of a property access on a union of objects?

The union of the property types, and only if the property exists in all members. Use in or a discriminant to narrow otherwise.

Q24. Can you narrow with the in operator? Limitations?

Yes, `"key" in obj` narrows to union members that have (or may have) that key. It does not work on primitives and can be fooled by optional properties.

Q25. What is the difference between a type assertion and a type guard?

An assertion (as T) only tells the compiler to trust you and has no runtime effect. A guard performs a runtime check and narrows safely.

Q26. How do you model a Result type safely?

`type Result<T,E> = { ok: true; value: T } | { ok: false; error: E };` Checking ok narrows to value or error.

Q27. How does any behave in unions and intersections?

`string | any` collapses to any, and `string & any` is also any. any absorbs other types, which is another reason to avoid it. By contrast, unknown absorbs in unions but disappears in intersections.

Q28. What is a type predicate vs an assertion function?

Predicate: `x is T` returns boolean and narrows in the true branch. Assertion function: `asserts x is T` throws on failure and narrows the code after the call.

Q29. How can you migrate a large JS codebase without spreading any?

Enable allowJs/checkJs gradually, turn on noImplicitAny per file, use unknown and narrow, add types at module boundaries, and use ESLint no-explicit-any / no-unsafe-* rules.

Q30. Can unions cause performance issues?

Very large unions (thousands of members) or complex distributive types can slow type checking. Simplify with generics, base types or branded types.

Output prediction

```
// 1. What is the type of r?
declare const x: string | number;
const r = typeof x === "string" ? x.length : x;   // number

// 2. Does this compile?
let v: string | number = "hi";
v.toUpperCase();     // Yes! v is narrowed to string by assignment

// 3. What is T?
type T = string | any;           // any
type U = string | unknown;       // unknown
type V = string & unknown;       // string
type W = string | never;         // string

// 4. Compile error?
function f(n: number | string) { return n.toFixed(); } // Error
```

11. Quick Cheat Sheet

Concept	Syntax / Example	Remember
Union	string | number	Either type; only shared members until narrowed
Literal union	'a' | 'b'	Replaces enums/magic strings
Discriminated union	{kind:'a'} | {kind:'b'}	Switch on kind, add never check
Narrowing tools	typeof, instanceof, in, ===, Array.isArray, predicates	Compiler tracks flow
any	let x: any	No checks, avoid
unknown	let x: unknown	Safe any; must narrow
never	() => never	Impossible value / exhaustiveness
Intersection	A & B	Combine types
Utility types	Exclude, Extract, NonNullable	Transform unions

Final takeaways

- Unions express **"one of these"** and keep your code type-safe through narrowing.
- Discriminated unions plus exhaustive checks eliminate whole classes of bugs.
- any is an **escape hatch**, not a default: reach for **unknown**, generics or unions first.
- Turn on **strict** mode and lint rules so unsafe any usage is caught early.